30-Day FreeRTOS Course for ESP32 Using ESP-IDF

(Day 10)

“Binary Semaphores”

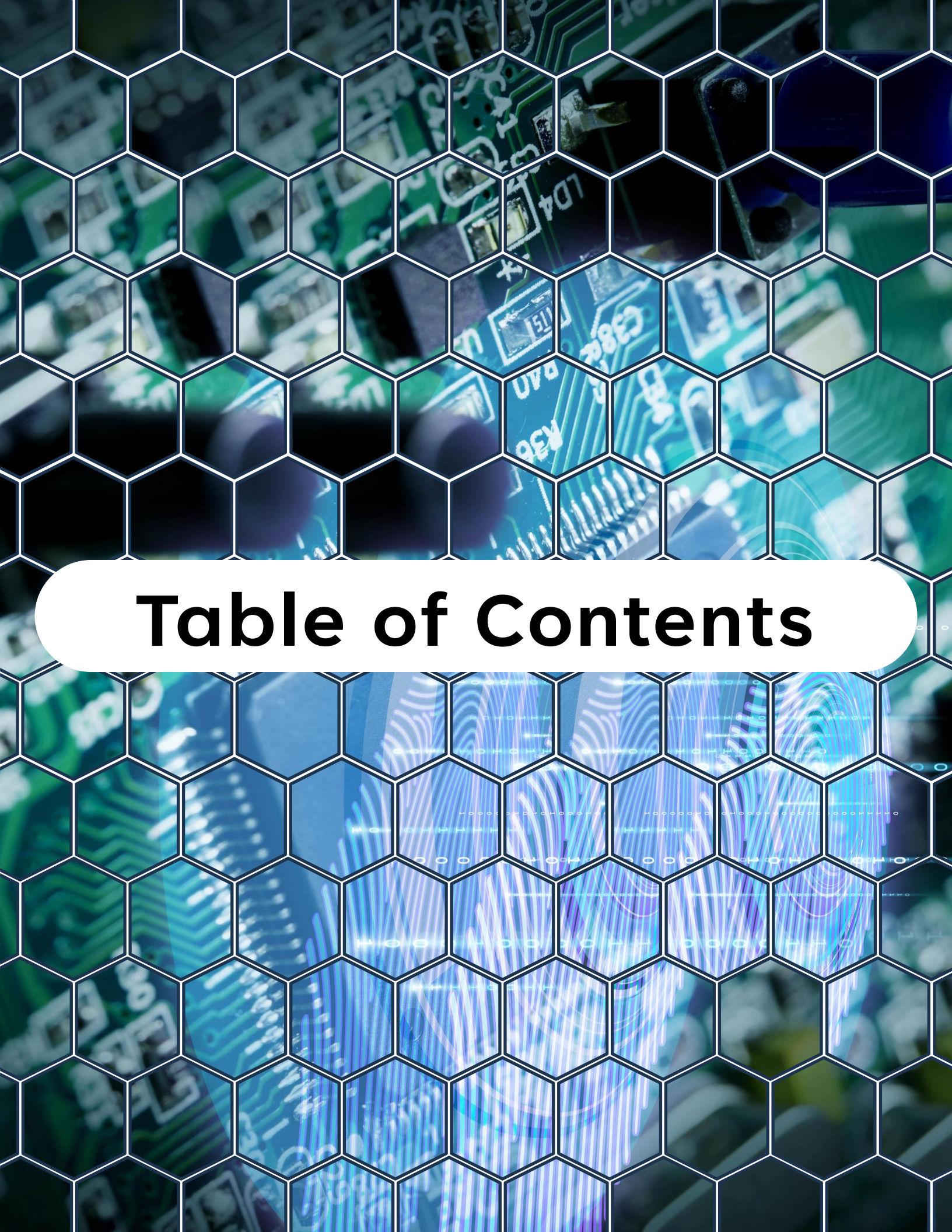
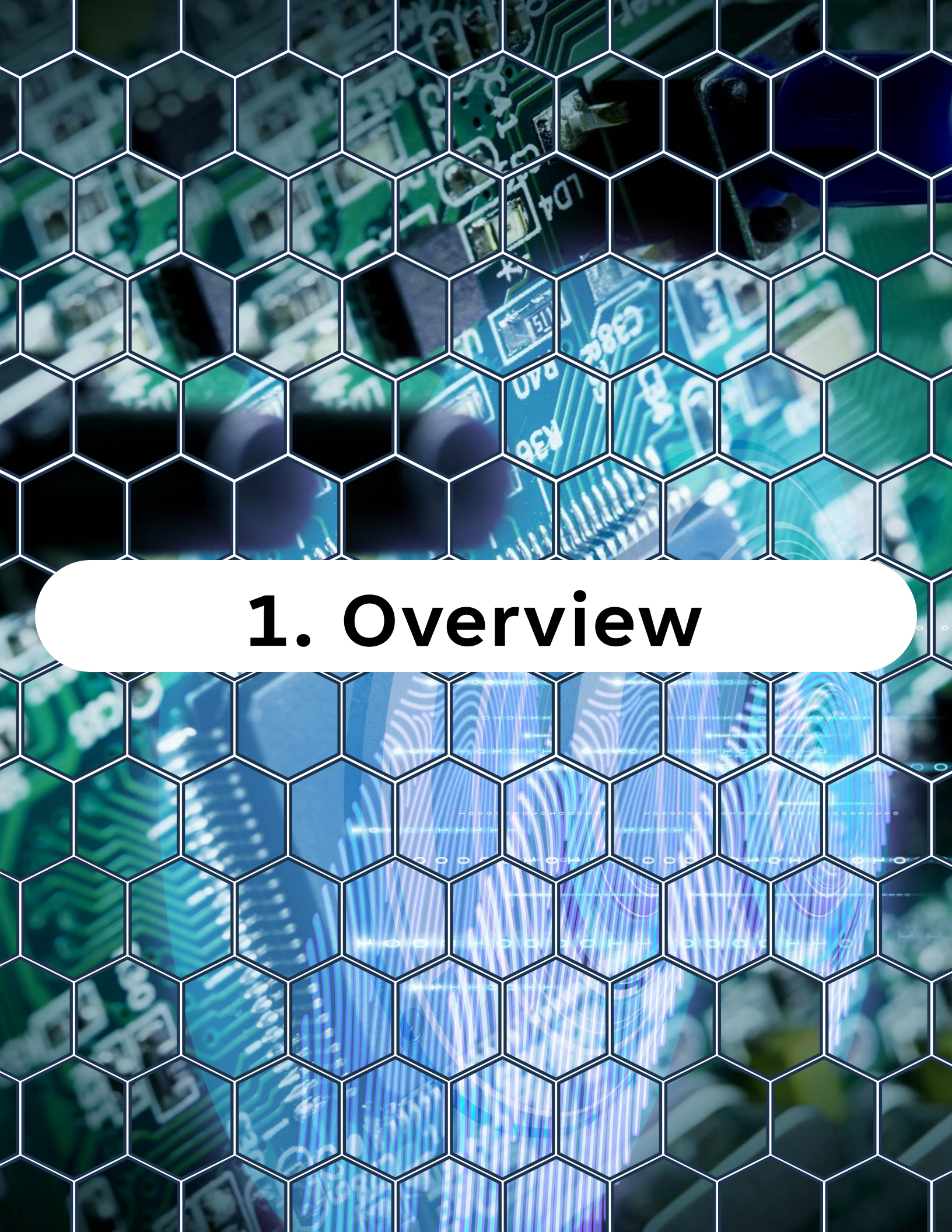


Table of Contents

Table of Contents



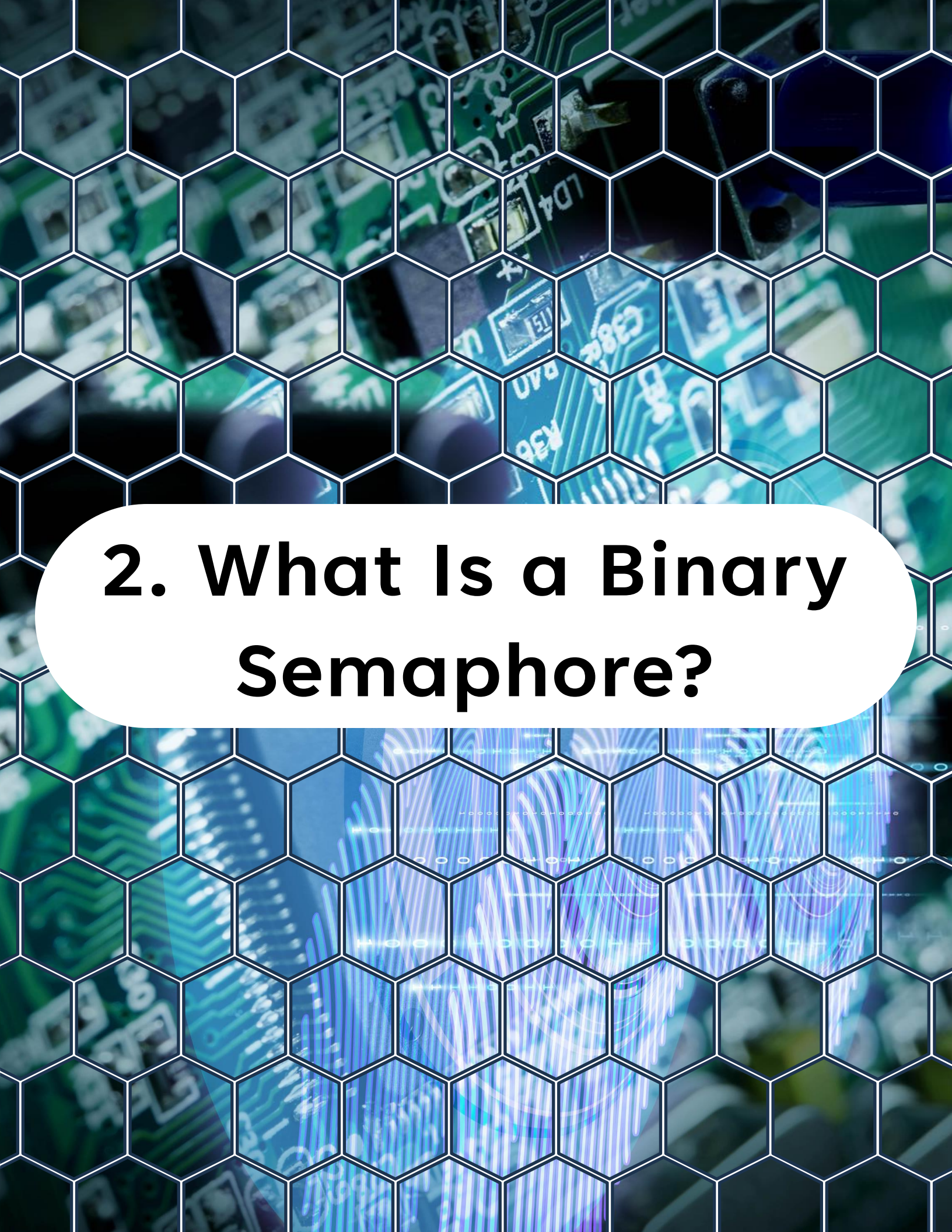
1. Overview

1. Overview

On **Day 10**, you'll learn:

- What **binary semaphores** are in FreeRTOS
- How they differ from queues and mutexes
- Typical use cases (task-to-task and ISR-to-task synchronization)
- How to create, take, and give semaphores
- A real example: **Button ISR → Binary Semaphore → Task**
- Best practices for semaphore usage

By the end of this lesson, you'll understand how to use binary semaphores for event signaling in ESP32 FreeRTOS applications.



2. What Is a Binary Semaphore?

2. What Is a Binary Semaphore?

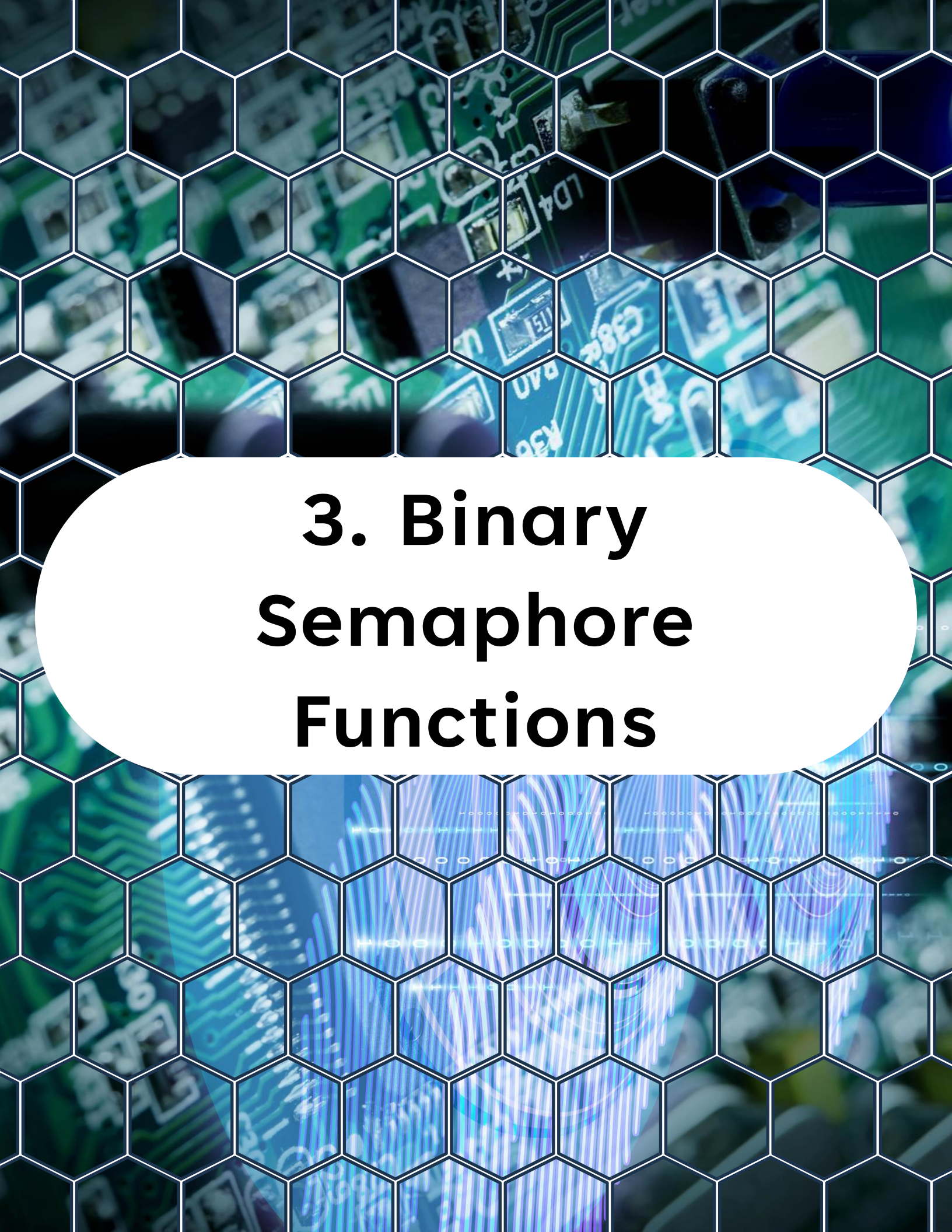
A binary semaphore is a simple synchronization tool with only two states:

- **Taken (0)** → unavailable
- **Given (1)** → available

It's typically used for **signaling events** between tasks or between ISRs and tasks. Unlike queues, semaphores don't store data — they simply notify that "something happened."

📌 **Key difference from mutex:**

- Mutexes enforce **ownership** (a task locks a resource and must unlock it).
- Binary semaphores are **event flags** — any task can give or take them.



3. Binary Semaphore Functions

3. Binary Semaphore Functions

Creating a Binary Semaphore

```
SemaphoreHandle_t xSemaphoreCreateBinary(void);
```

Giving (releasing) a Semaphore

```
xSemaphoreGive(SemaphoreHandle_t xSemaphore);
```

ISR-safe version

```
xSemaphoreGiveFromISR(SemaphoreHandle_t xSemaphore,
                      BaseType_t *pxHigherPriorityTaskWoken);
```

Taking (waiting) a Semaphore

```
xSemaphoreTake(SemaphoreHandle_t xSemaphore,
               TickType_t xTicksToWait);
```

- Blocks the task until the semaphore becomes available, or until timeout expires.
- If `xTicksToWait = portMAX_DELAY`, the task waits indefinitely.

4. Common Use Cases

4. Common Use Cases

- ✅ ISR-to-task signaling – e.g., when a button is pressed, an ISR gives a semaphore to unblock a task.
- ✅ Task-to-task synchronization – one task signals another that an event is complete.
- ✅ Resource gating – ensure only one task at a time proceeds when a semaphore is available (though mutexes are better for this).

5. Example – Button ISR with Binary Semaphore

5. Example – Button ISR with Binary Semaphore

```c
/**
 * @file day10_button_semaphore.c
 * @brief ESP-IDF example: button ISR signals a binary semaphore, task waits and handles the press.
 *
 * The GPIO interrupt service routine (ISR) gives a binary semaphore using
 * xSemaphoreGiveFromISR(). A FreeRTOS task blocks on that semaphore via
 * xSemaphoreTake() and logs the button press, where debouncing or actions can be added.
 */

#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/semphr.h"
#include "driver/gpio.h"
#include "esp_log.h"

#define BUTTON_GPIO 0   // BOOT button on many ESP32 boards
#define TAG "DAY10"

/** @brief Binary semaphore signaled from the ISR and taken by the button task. */
SemaphoreHandle_t button_semaphore;

/**
 * @brief GPIO interrupt service routine; gives the semaphore to wake the waiting task.
 *
 * Called on the configured edge for BUTTON_GPIO. Uses xSemaphoreGiveFromISR() and,
 * if a higher-priority task is unblocked, requests an immediate context switch.
 *
 * @param arg Optional ISR argument (unused).
 */
static void IRAM_ATTR button_isr_handler(void *arg) {
    (void)arg; // Unused
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    // Give semaphore from ISR
    xSemaphoreGiveFromISR(button_semaphore, &xHigherPriorityTaskWoken);

    // If a higher-priority task was woken, yield immediately
    if (xHigherPriorityTaskWoken) {
        portYIELD_FROM_ISR();
    }
}

/**
 * @brief Task that blocks on the semaphore and handles button presses.
 *
 * Waits indefinitely for the semaphore given by the ISR. When taken, logs the
 * event and serves as a placeholder for debouncing or application logic.
 *
```

5. Example – Button ISR with Binary Semaphore

```
/**
 * @brief Task that blocks on the semaphore and handles button presses.
 *
 * Waits indefinitely for the semaphore given by the ISR. When taken, logs the
 * event and serves as a placeholder for debouncing or application logic.
 *
 * @param pvParameter Optional task parameter (unused).
 */
void button_task(void *pvParameter) {
    (void)pvParameter; // Unused
    while (1) {
        if (xSemaphoreTake(button_semaphore, portMAX_DELAY)) {
            ESP_LOGI(TAG, "Button pressed!");
            // Do work here (debounce, action, etc.)
        }
    }
}

/**
 * @brief Application entry point: sets up GPIO, ISR, semaphore, and the task.
 *
 * Creates a binary semaphore, configures BUTTON_GPIO as input with pull-up and
 * falling-edge interrupt, installs/attaches the ISR, and starts the button task.
 */
void app_main() {
    // Create binary semaphore
    button_semaphore = xSemaphoreCreateBinary();
    if (button_semaphore == NULL) {
        ESP_LOGE(TAG, "Failed to create semaphore");
        return;
    }

    // Configure button pin
    gpio_config_t io_conf = {
        .intr_type = GPIO_INTR_NEGEDGE,  // Falling edge
        .mode = GPIO_MODE_INPUT,
        .pin_bit_mask = (1ULL << BUTTON_GPIO),
        .pull_up_en = GPIO_PULLUP_ENABLE,
        .pull_down_en = GPIO_PULLDOWN_DISABLE
    };
    gpio_config(&io_conf);

    // Install ISR service and attach handler
    gpio_install_isr_service(0);
    gpio_isr_handler_add(BUTTON_GPIO, button_isr_handler, NULL);

    // Create task to handle button events
    xTaskCreate(button_task, "ButtonTask", 2048, NULL, 10, NULL);
}
```

5. Example – Button ISR with Binary Semaphore

Expected Behavior:

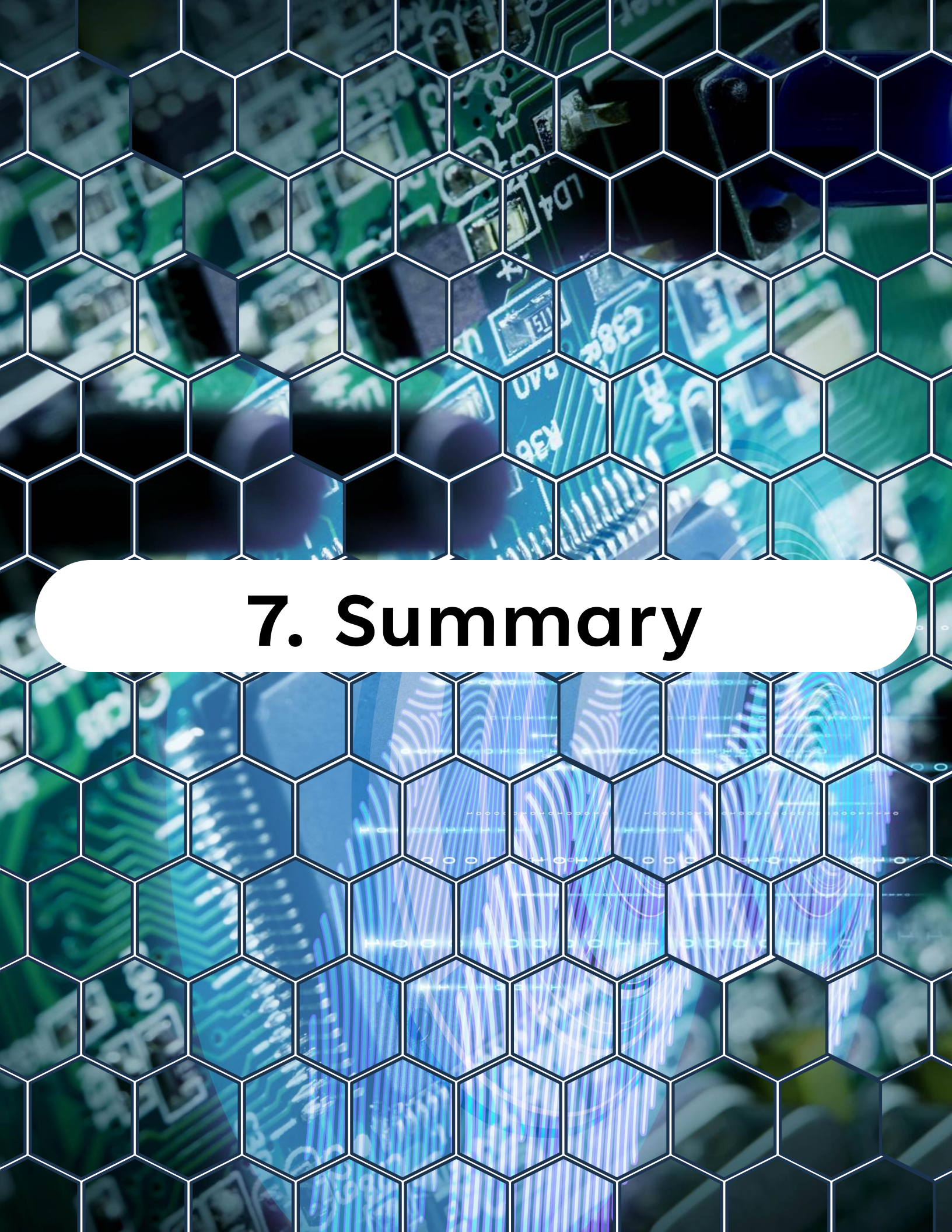
- Press the button on GPIO0 (BOOT button on many ESP32 boards).
- The ISR runs and gives the semaphore.
- The button_task unblocks and prints:

```
I (1234) DAY10: Button pressed!
```

6. Best Practices

6. Best Practices

- ✅ Always use **ISR-safe functions** (`xSemaphoreGiveFromISR`).
- ✅ Keep ISRs short: don’t process events inside ISR, just signal.
- ✅ Use **binary semaphores for signaling events**, not for protecting data (mutexes are better for that).
- ✅ If multiple tasks need to react to the same event, consider **event groups** (Day 15).
- ✅ Initialize semaphores immediately in `app_main()` or before creating tasks.



7. Summary

7. Summary

- A **binary semaphore** is a synchronization tool with only two states (0/1).
- Used mainly for **event signaling** between ISR ↔ task or task ↔ task.
- Use `xSemaphoreGiveFromISR()` inside ISRs to unblock tasks.
- Keep ISRs lean — let tasks do the heavy lifting.

8. Challenge for Today

8. Challenge for Today

Extend today's project by:

1. Adding a **second task** that also waits on the semaphore.
2. Observe that **only one task** unblocks per event (semaphore is not a broadcast).
3. Modify the project to use an **event group** instead (coming on Day 15) to notify multiple tasks simultaneously.